SPLITSTACK

SplitStack Refurbishment - Prompt Log

All prompts used to analyse, plan and rebuild SplitStack, plus the planned prompts that complete the project.

Student: Mahaprabhu K
Assistant: Claude Code (Claude Opus 5.5)
Repository: https://github.com/MahaprabhuK/SplitStack
Executed prompts: 47
Planned prompts: 2
Last updated: 2026-10-17

How to read this log

Part A lists every step already carried out, in order. "User prompt" entries are the student's own messages, quoted verbatim; "Working prompt" entries are the instructions executed for that step, written so they can be reused. Part B lists the planned prompts that complete Review 2, Review 3 and the wrap-up; they have not been run yet and are marked "Planned prompt".

Stages

Stage	Scope	Prompts
Planning	Analysis of the existing project, audit report review, gap analysis and refurbishment plan	4
Phase 0	Repository setup, hygiene and documentation tooling	2
Review 1	HTML + CSS - static, semantic pages and a token-based design system	6
Review 2	JavaScript - vanilla ES modules, interactivity, client-side algorithms	16
Review 3	Node.js + MongoDB - Express REST API, Mongoose models, authentication, OCR	11
Wrap-up	Documentation, cleanup and deployment	10

Summary

#	Stage	Title	Type	Status
1	Planning	Initial request from the student	User prompt	Done
2	Planning	Analyse the repository	Working prompt	Done
3	Planning	Analyse the audit report	Working prompt	Done
4	Planning	Gap analysis and refurbishment plan	Working prompt	Done
5	Phase 0	Branching and repository hygiene	Working prompt	Done
6	Phase 0	Prompt-log tooling (this document)	Working prompt	Done
7	Review 1	CSS design system with cascade layers	Working prompt	Done
8	Review 1	Static semantic HTML pages (25 pages)	Working prompt	Done
9	Review 1	Local static server for previewing the client	Working prompt	Done
10	Review 1	Validate HTML and fix issues	Working prompt	Done
11	Review 1	Visual QA and layout bug fixes	Working prompt	Done
12	Review 1	Client README and Review 1 checkpoint commit	Working prompt	Done
13	Wrap-up	Request: GitHub repository, website link and PDF of prompts	User prompt	Done
14	Wrap-up	PDF export of the prompt log, including planned prompts	Working prompt	Done
15	Wrap-up	Create the GitHub repository and push the code	Working prompt	Done
16	Wrap-up	Request: hand the project over to a teammate's agent	User prompt	Done
17	Wrap-up	Push the refurbished code to GitHub main	Working prompt	Done
18	Wrap-up	Request: push the code and deploy the website	User prompt	Done
19	Wrap-up	Deploy the client to GitHub Pages from a gh-pages branch	Working prompt	Done
20	Wrap-up	Request: handover prompt PDF for a teammate's agent	User prompt	Done

#	Stage	Title	Type	Status
21	Wrap-up	Handover prompts PDF	Working prompt	Done
22	Review 2	Request: proceed with Review 2 Step P1	User prompt	Done
23	Review 2	JavaScript architecture and app shell	Working prompt	Done
24	Review 2	Request: proceed with Review 2 Step P2	User prompt	Done
25	Review 2	Client data layer backed by localStorage	Working prompt	Done
26	Review 2	Request: proceed with Review 2 Step P3	User prompt	Done
27	Review 2	Render pages from data	Working prompt	Done
28	Review 2	Request: proceed with Review 2 Step P4	User prompt	Done
29	Review 2	Forms, validation and CRUD flows	Working prompt	Done
30	Review 2	Request: proceed with Review 2 Step P5	User prompt	Done
31	Review 2	Group splitting and debt simplification	Working prompt	Done
32	Review 2	Request: proceed with Review 2 Step P6	User prompt	Done
33	Review 2	Charts, budgets, forecast, insights, search and notifications	Working prompt	Done
34	Review 2	Request: proceed with Review 2 Step P7	User prompt	Done
35	Review 2	Shopping checklist and receipt scan preview	Working prompt	Done
36	Review 2	Request: proceed with Review 2 Step P8	User prompt	Done
37	Review 2	Review 2 QA, final audit and regression verification	Working prompt	Done
38	Review 3	Request: proceed with Review 3 Step P9	User prompt	Done
39	Review 3	Express server foundation	Working prompt	Done
40	Review 3	Request: proceed with Review 3 Step P10	User prompt	Done
41	Review 3	MongoDB connection and Mongoose data models	Working prompt	Done
42	Review 3	Request: proceed with Review 3 Step P11	User prompt	Done
43	Review 3	Authentication and REST API endpoints	Working prompt	Done
44	Review 3	Request: proceed with Review 3 Step P12	User prompt	Done
45	Review 3	Receipt upload and server-side OCR	Working prompt	Done
46	Review 3	Request: proceed with Review 3 Step P13	User prompt	Done
47	Review 3	Frontend API integration	Working prompt	Done
P1	Review 3	Seed data and automated tests	Planned prompt	Planned
P2	Wrap-up	Remove legacy code, document and deploy	Planned prompt	Planned

Part A - Executed prompts

Planning - Analysis of the existing project, audit report review, gap analysis and refurbishment plan

#1 Initial request from the student

User prompt | 2026-10-14

PROMPT

```
i have a project in my web technology course and we have made a website. i want to check everything and refurbish if needed. the project shd have these things. html + css, javascript, node js , mangodbs. for review 1-html+css, for review 2 javascript. for review 3 the rest two. i have attached a report of the existing project. i want you to analyse the repositry and then the audit report. and give a good plan for the refurshing if needed. during refurbishing i want you to document the prompts you'll use to build the project save it as a log document(word)
```

PURPOSE

Defines the goal: verify the project meets the course stack (HTML+CSS, JavaScript, Node.js, MongoDB) across three reviews, refurbish it, and log all prompts in a Word document.

FILES

SplitStack_Comprehensive_Technical_Audit.pdf (input)

RESULT

Task accepted; analysis started.

VERIFICATION

-

#2 Analyse the repository

Working prompt | 2026-10-14

PROMPT

```
Inventory the SplitStack repository: list every tracked file, read the git history, the dependency manifests (requirements.txt / package.json), the base HTML layout, the CSS design tokens and all JavaScript (main.js plus inline <script> blocks in templates). Identify the actual technology stack and compare it with the course requirement of HTML+CSS, JavaScript, Node.js and MongoDB.
```

PURPOSE

Establish ground truth about what the code really is, independent of the audit report.

FILES

app.py, requirements.txt, templates/**, static/css/style.css, static/js/main.js, .gitignore

RESULT

Stack is Python 3 + Flask 3.1 + SQLite with Jinja2 templates; no Node.js, no MongoDB, no package.json. Frontend JS is ~140 lines of jQuery. 25 Jinja templates cannot render without Flask. 4 user-uploaded receipt images are committed to git.

VERIFICATION

File listing, git log, grep for <script> tags, manual reading of base.html and main.js.

#3 Analyse the audit report

Working prompt | 2026-10-14

PROMPT

```
Extract the text of SplitStack_Comprehensive_Technical_Audit.pdf (61 pages) and summarise its findings: technology stack, architecture, route inventory, database schema, security findings (SEC-01..SEC-07), test results, technical debt and the five-phase refurbishment roadmap (Phases A-E). Note anything the audit does not cover relative to the course requirements.
```

PURPOSE

Reuse the audit's verified findings and identify its blind spots.

FILES

SplitStack_Comprehensive_Technical_Audit.pdf

RESULT

Audit confirms 48 routes, 7 blueprints, 15 SQLite tables, 47/47 tests passing. Key findings kept: incomplete light theme, --text-dim contrast 3.8:1 (fails WCAG AA), tables scroll on mobile, search/notification bell not wired, no rate limiting, no security headers, ephemeral upload storage. Blind spot: the audit never mentions that Node.js and MongoDB are required, so its Flask-specific roadmap (SQLite WAL, Flask-Limiter, PostgreSQL) does not satisfy Review 3.

VERIFICATION

Text extracted with pypdf; sections 2, 12-17 and Volume VI read.

#4 Gap analysis and refurbishment plan

Working prompt | 2026-10-14

PROMPT

```
Produce a refurbishment plan that maps the work onto the three course reviews: Review 1 must be demonstrable with only HTML+CSS, Review 2 must add substantial vanilla JavaScript on top of the same pages, and Review 3 must replace the backend with Node.js (Express) and MongoDB while keeping every existing feature. Fold in the still-relevant audit findings. Ask the student only about decisions that are genuinely theirs: backend strategy, where MongoDB runs, feature scope, and whether to keep jQuery.
```

PURPOSE

Turn the analysis into an ordered plan with clear review checkpoints.

FILES

docs/REFURBISH_PLAN.md

RESULT

Plan: Phase 0 setup -> Review 1 static pages + design system -> Review 2 vanilla JS with a localStorage data layer -> Review 3 Express + Mongoose REST API, with the frontend data layer switched to fetch(). Student's answers: 'do the most optimal method' for the backend (=> replace Flask with Node+MongoDB, keep Flask on a legacy branch), MongoDB Atlas free tier, keep every feature including OCR (via tesseract.js), vanilla JS modules instead of jQuery. Student also asked: 'just make sure you log everything'.

VERIFICATION

Decisions confirmed through a multiple-choice question in chat.

Phase 0 - Repository setup, hygiene and documentation tooling

#5 Branching and repository hygiene

Working prompt | 2026-10-14

PROMPT

```
Preserve the current Flask application on a local 'legacy' branch and create a 'refurbish'
working branch. Stop tracking the four user-uploaded receipt images (hash-named files in
uploads/receipts/) while keeping the test_invoice_* samples, deduplicate .gitignore, and add
Node-specific ignores (node_modules, coverage, .env). Create the client/ folder structure
(pages, css, js, assets).
```

PURPOSE

Protect the original work, remove private data from version control, and prepare the new layout.

FILES

.gitignore, uploads/receipts/* (untracked), client/

RESULT

Branches legacy and refurbish created; 4 receipts untracked (still on disk); .gitignore rewritten.

VERIFICATION

git status shows the 4 receipts as deleted from the index and no longer as untracked files.

#6 Prompt-log tooling (this document)

Working prompt | 2026-10-14

PROMPT

```
Create a Word prompt log that is easy to keep up to date: store every prompt as an entry in
docs/prompt_log.json (id, date, stage, title, type, prompt, purpose, files, result,
verification) and write a Node script docs/tools/build-prompt-log.js using the 'docx' npm
package that renders a title page, a stage overview, a summary table of all entries, and one
detailed section per entry with the prompt shown verbatim. Add an 'npm run log' script.
```

PURPOSE

Satisfy the requirement to document every prompt as a Word document, regenerated after each step.

FILES

package.json, docs/prompt_log.json, docs/tools/build-prompt-log.js, docs/SplitStack_Prompt_Log.docx

RESULT

Log generator working; document regenerated after every subsequent step.

VERIFICATION

npm run log produces the .docx; contents read back with python-docx.

Review 1 - HTML + CSS - static, semantic pages and a token-based design system

#7 CSS design system with cascade layers

Working prompt | 2026-10-14

PROMPT

```
Rebuild SplitStack's stylesheet as a token-based design system split into files and ordered with CSS cascade layers: tokens.css (colours, type scale, spacing, radii, shadows; dark default plus a light theme via prefers-color-scheme and a [data-theme] attribute for later JavaScript), base.css (reset, typography, focus ring, skip link, visually-hidden), layout.css (app shell with sticky sidebar and topbar, grids, auth and landing layouts, responsive breakpoints at 1100/900/640px, CSS-only mobile drawer using a hidden checkbox and :has()), components.css (cards, buttons, badges, category tags via a --cat custom property, stat cards, budget meters, forms with :user-invalid feedback, segmented radios, switches, tables that become stacked cards under 640px using data-label, alerts, empty states, lists, CSS-only bar and conic-gradient doughnut charts, transfer cards, checklist, file drop zone), pages.css and utilities.css. main.css imports them with @layer. Fix the audit findings: muted text must reach at least 4.5:1 contrast, the light theme must be complete, and tables must not scroll sideways on mobile.
```

PURPOSE

Review 1 requires a clean, well-structured CSS codebase; this also fixes audit items SEC-06 (contrast), B.1 (light theme) and B.2 (responsive tables).

FILES

client/css/main.css, client/css/tokens.css, client/css/base.css, client/css/layout.css, client/css/components.css, client/css/pages.css, client/css/utilities.css

RESULT

Six layered stylesheets. --text-dim raised from #64748b (3.8:1) to #8794a9 (about 5.6:1) on dark and #5d6879 on light surfaces.

VERIFICATION

Rendered in the browser at 1366px and 375px; computed styles inspected.

#8 Static semantic HTML pages (25 pages)

Working prompt | 2026-10-14

PROMPT

```
Convert every Jinja template of the Flask app into a standalone static HTML5 page under client/ that opens without a server-side framework, using realistic sample data in Indian rupees for one demo user (Priya Raman) as of 28 Oct 2026. Pages: landing (index.html), login, register, 404, dashboard, analytics, expenses, add expense, edit expense, income, log income, budgets, recurring payments, categories, groups, create group, group detail, add shared expense, settle up, shopping lists, new list, list detail, receipt scan, receipt confirm, profile. Every app page shares the same sidebar, topbar (search, notifications, avatar) and footer. Use semantic landmarks (header, nav, main, aside, section, footer), a skip link, breadcrumbs, labelled forms with HTML5 validation (required, min, max, step, pattern, type=email/date/number), tables with captions and scope, time elements, aria-current for the active link, role=progressbar with values for budget meters, and a text alternative for every chart. Keep the sample numbers internally consistent (category totals, group balances, the debt-simplification plan). Avoid inline styles except CSS custom properties such as --value or --cat. Generate the repeated layout with a throwaway script so that the committed output is plain HTML.
```

PURPOSE

Deliver the Review 1 HTML+CSS milestone: every screen of SplitStack, demonstrable with only HTML and CSS.

FILES

client/index.html, client/pages/*.html (24 pages), client/assets/favicon.svg, client/assets/sample-receipt.webp

RESULT

25 static pages. CSS-only interactions: mobile drawer (checkbox + :has), equal/custom split toggle (:has(#split-custom:checked)), purchased items struck through (:has(input:checked)), click-or-drop file input.

VERIFICATION

See the validation entry.

#9 Local static server for previewing the client

Working prompt | 2026-10-14

PROMPT

```
Add a dependency-free Node static file server (scripts/serve-client.js) that serves client/ on http://localhost:5500, refuses paths outside client/, falls back to pages/404.html, and is started with "npm run client". Opening pages straight from disk also works, but Review 2 ES modules will need an http:// origin.
```

PURPOSE

Preview the site reliably now and prepare for JavaScript modules in Review 2.

FILES

scripts/serve-client.js, package.json, .claude/launch.json

RESULT

npm run client serves the site.

VERIFICATION

Pages loaded in the in-app browser at localhost:5500.

#10 Validate HTML and fix issues

Working prompt | 2026-10-14

PROMPT

```
Install html-validate as a dev dependency with a project config (.htmlvalidate.json) based on html-validate:recommended, run it on client/**/*.html and fix every error: unescaped ampersands, a search form without a submit button, unnamed duplicate landmarks, aria-label on a plain div, a form without a submit button, redundant ARIA roles. Allow shared names for checkbox groups and the custom progressbar (needed for over-100% budget states). Then check that every relative href/src resolves to a real file.
```

PURPOSE

Show that the Review 1 markup is standards-compliant and accessible.

FILES

.htmlvalidate.json, package.json, client/**/*.html

RESULT

First run: 100 errors. After fixes: 0 errors across 25 pages; 0 broken internal links.

VERIFICATION

npx html-validate "client/**/*.html" exits 0.

#11 Visual QA and layout bug fixes

Working prompt | 2026-10-14

PROMPT

```
Open the pages in the browser at desktop (1366px) and mobile (375px) widths, look for layout bugs and fix them in the CSS. Then load every page in a 375px and a 1280px iframe and report any element that overflows the viewport horizontally.
```

PURPOSE

Catch rendering problems that a validator cannot see.

FILES

client/css/layout.css, client/css/components.css

RESULT

Fixed: (1) the scrim <label> occupied a grid cell on desktop and pushed the main column below the sidebar; (2) the mobile menu button showed on desktop and the search icon overlapped the placeholder, because the components layer overrides the layout layer; those elements now have layout-only classes; (3) table dates wrapped onto two lines.

VERIFICATION

Screenshots at both widths; scripted overflow check across all pages.

#12 Client README and Review 1 checkpoint commit

Working prompt | 2026-10-14

PROMPT

```
Write client/README.md explaining how to view the site (npm run client), the folder structure, and a table of the CSS techniques used, with the file each one is in, so the Review 1 demo can point to them. Add an "npm run lint:html" script. Re-run validation, regenerate the prompt log, and commit Review 1 on the refurbish branch.
```

PURPOSE

Make Review 1 easy to present and record a clean checkpoint.

FILES

client/README.md, package.json, docs/SplitStack_Prompt_Log.docx

RESULT

Review 1 committed. Next: Review 2 (vanilla JavaScript modules).

VERIFICATION

npm run lint:html passes; git log shows the Review 1 commit.

Review 2 - JavaScript - vanilla ES modules, interactivity, client-side algorithms

#22 Request: proceed with Review 2 Step P1

User prompt | 2026-10-16

PROMPT

```
Proceed with P1 — JavaScript architecture and app shell — from the handover PDF. Preconditions: Confirm the current branch is refurbish and the working tree has no unexpected changes. Preserve the untracked docs/audit/ directory. Install the root npm dependencies using the existing package.json. For this PowerShell session, set $env:PY_PYTHON3 = "3.12" so py -3 resolves correctly. Check whether ReportLab is installed; if not, install it for Python 3.12. Do not change Git remotes, push to a different repository, or alter package scripts to work around environment issues. Implement only P1 as specified in SplitStack_Handover_Prompts.pdf. Read the relevant HTML and CSS before editing. Preserve the existing design and no-JavaScript fallback. Follow the PDF's accessibility, theme, navigation, toast, confirmation-dialog, logging, and verification requirements. Run npm run lint:html and the P1 manual checks you can perform. Record anything you could not verify honestly. Before committing, show me the proposed changed-file list and verification results. Do not commit or push until I approve. Do not start P2.
```

PURPOSE

Authorize execution of Step P1 with strict preconditions and manual approval gate.

RESULT

Confirmed clean refurbish branch, installed dependencies and ReportLab, and executed Step P1 implementation.

VERIFICATION

npm run lint:html, automated unit tests, and serve-client HTTP validation.

#23 JavaScript architecture and app shell

Working prompt | 2026-10-16

PROMPT

```
Step P1 (Review 2): JavaScript architecture and app shell. Add vanilla JavaScript to the Review 1 pages as native ES modules. 1. Add data-page="<page-name>" to the <body> of index.html and every file in client/pages/ (e.g. data-page="dashboard", data-page="expense-form"). Load one entry script at the end of each page: <script type="module" src="../js/app.js"></script> (index.html: src="js/app.js"). 2. Create client/js/ with: - app.js: entry point. Reads document.body.dataset.page, initialises the shared UI, then dynamically imports ./pages/<page>.js if it exists and calls its exported init(). A missing page module is not an error. - core/dom.js: qs, qsa, on, delegate, createEl helpers. - core/format.js: formatINR(paise) using Intl.NumberFormat('en-IN', {style:'currency', currency:'INR'}), formatDate / formatRelativeDate helpers. - ui/theme.js: dark / light / system toggle saved in localStorage, applied to <html data-theme>. Add a tiny inline script in each page's <head> that applies the saved theme before first paint (no flash). Wire it to a theme button in the topbar (add the button if missing, matching existing button styles). - ui/nav.js: replace the CSS checkbox drawer with a real <button aria-expanded aria-controls="sidebar">. Close on Escape, scrim click and nav-link click; trap focus inside while open; return focus to the button when closed. Keep the page usable without JS. - ui/toast.js: accessible toasts (role="status", aria-live="polite"), auto-dismiss, dismiss button. - ui/confirm.js: a <dialog>-based confirm that replaces window.confirm for every delete button/form. 3. Add client/js/package.json containing {"type": "module"} so Node (used for unit tests later) treats these files as ES modules. 4. All wrapped in try/catch where localStorage is touched (private windows can throw).
```

PURPOSE

Establish the client JavaScript ES module foundation, theme switching, accessible drawer, toast notifications, and confirm dialog.

FILES

client/js/package.json, client/js/app.js, client/js/core/dom.js, client/js/core/format.js, client/js/ui/theme.js, client/js/ui/nav.js, client/js/ui/toast.js, client/js/ui/confirm.js, client/css/layout.css, client/css/components.css, client/index.html, client/pages/*.html

RESULT

Created client/js architecture with DOM/format utilities, theme toggle (dark/light/system with head flash-prevention script), accessible mobile drawer with focus trap and ARIA attributes, toast notifications, and dialog-based confirm. Added data-page, inline theme scripts, theme buttons, and app.js script tags to all 25 HTML pages while preserving no-JS CSS fallbacks.

VERIFICATION

html-validate passes with 0 errors across client/**/*.html; node automated assertions pass for formatINR, formatDate, formatRelativeDate, HTML structure, and CSS layer definitions; static client preview verified over HTTP on localhost:5500.

#24 Request: proceed with Review 2 Step P2

User prompt | 2026-10-16

PROMPT

```
Proceed to Step P2: Client data layer backed by localStorage, following the project handover PDF. 1. Read the P2 instructions in the handover PDF and inspect the committed P1 architecture before making changes. 2. Implement only the P2 data layer. Keep storage and data operations separate from UI rendering and page-specific behavior. 3. Use a consistent, versioned storage structure and handle missing, malformed, or unsupported stored data safely. Avoid silently overwriting recoverable user data. 4. Follow the existing JavaScript module conventions and use the existing utilities where appropriate. 5. Add focused automated tests for storage initialization, read/write behavior, invalid data, and error handling where practical. 6. Run the relevant tests and npm run lint:html. Report exactly what passed and failed. 7. Update the prompt log and regenerate its DOCX/PDF using the existing project workflow. 8. Review the final diff for scope and regressions. Preserve docs/audit/ and all legacy Flask files. 9. Commit the completed P2 work on refurbish only after the checks pass. Use the commit message specified by the handover PDF, if one is provided; otherwise use P2: Client data layer backed by localStorage. Do not begin P3 or P4, implement Express/MongoDB, push to origin or upstream, or delete legacy files. Finish by reporting the changed files, tests, commit hash, and any remaining limitations.
```

PURPOSE

Authorize execution of Step P2 to establish the client data layer backed by localStorage with realistic seed data and API contracts.

RESULT

Implemented store.js with versioned schema and fallback, seed.js with October 2026 and historical data, and api.js with 58 async API functions matching the future REST API.

VERIFICATION

26 automated tests passing in tests/client/data-layer.test.mjs; npm run lint:html passing with 0 errors across all HTML pages.

#25 Client data layer backed by localStorage

Working prompt | 2026-10-16

PROMPT

```
Step P2 (Review 2): client data layer backed by localStorage. Create client/js/data/ with: - store.js: keeps collections (users, categories, expenses, income, budgets, recurring, groups, sharedExpenses, settlements, shoppingLists, receipts, insights) in localStorage under one versioned key (e.g. "splitstack:v1"), generates ids (crypto.randomUUID), handles a missing/corrupt store by reseeding, and has a reset() used by a "Reset demo data" button on the profile page. - seed.js: the SAME sample data the Review 1 pages show. Read the HTML pages (dashboard, expenses, income, budgets, recurring, categories, groups, group-detail, settlements, shopping, shopping-detail, profile) and transcribe their data: demo user Priya Raman, October 2026, the groups (incl. Goa Trip) and their members, the shopping lists and items. Add a few earlier months so analytics has history. Amounts in integer paise. Demo login: demo@splitstack.app / Demo@1234 (show this on login.html). - api.js: async functions with the shape the future REST API will have, grouped as auth (register, login, logout, currentUser, updateProfile, changePassword), expenses and income (list with filter/sort/pagination, get, create, update, remove), categories, budgets, recurring (incl. toggle), groups (list, get, create, addMember, removeMember, leave, sharedExpenses, balances, settlements, recordSettlement), shopping (lists, items, toggleItem, convertToExpense), analytics (monthlySummary, byCategory, forecast, insights) and search(query). Every function returns a Promise, validates its input and throws an ApiError {field, message, status} for bad input, missing records or records owned by someone else. Passwords: hash with crypto.subtle SHA-256 + salt (demo only; note in a comment that the server uses bcrypt). Page code must import ONLY from api.js, never from store.js. - Write a short JSDoc block above each function describing arguments and return value. These docs become the contract the Review 3 REST API must match.
```

PURPOSE

Create a complete, modular, resilient client data layer with realistic seed data and REST-shaped async API functions, preparing the application for full interactivity without requiring a backend server.

FILES

client/js/data/store.js, client/js/data/seed.js, client/js/data/api.js, client/pages/login.html, client/README.md, tests/client/data-layer.test.mjs

RESULT

Implemented store.js with versioning (splitstack:v1), corruption recovery, and in-memory fallback; seed.js with Priya Raman sample data in integer paise across all collections and historical months; api.js with 58 JSDoc-documented async functions covering auth, CRUD, group splitting balances, shopping lists, and analytics; updated login.html with demo credentials callout; documented data layer in client/README.md; added automated tests.

VERIFICATION

Node.js test suite with 26/26 tests passing covering storage recovery, authentication, integer-paise validation, CRUD, group balances, and shopping list conversions; npm run lint:html passing with 0 errors.

#26 Request: proceed with Review 2 Step P3

User prompt | 2026-10-16

PROMPT

```
Proceed to Step P3: Render pages from data, following the project handover PDF. Before coding, inspect the current refurbish branch and read the exact P3 requirements. Build on the committed P1 architecture and P2 data layer, including the fixes in commit 89c3081. Requirements: 1. Implement only P3: connect the existing HTML pages to client/js/data/api.js and render their content from returned data. 2. Page modules must import data operations only from api.js, never directly from store.js or seed.js. 3. Preserve the existing design, responsive layouts, accessibility, and useful empty states. 4. Handle loading, errors, and missing records gracefully. Do not show hardcoded success messages for operations that have not occurred. 5. Keep static HTML fallbacks usable where practical. Avoid duplicating business logic across page modules. 6. Follow the handover's page/module organization and update documentation only where needed for P3. 7. Add focused tests for rendering and error states where practical. Run the relevant tests and npm run lint:html. 8. Update the prompt log and regenerate its DOCX/PDF using the existing workflow. 9. Review the diff and report which pages are connected, which remain static, test results, and known limitations. Preserve docs/audit/ and all legacy Flask files. Do not begin P4 forms/CRUD, Express/MongoDB, or deployment work. Do not push to origin or upstream. Commit P3 locally on refurbish only after the checks pass, using the commit message specified in the handover or P3: render pages from data if none is specified.
```

PURPOSE

Authorize execution of Step P3 to connect the existing HTML pages to client/js/data/api.js and dynamically render their content from data.

RESULT

Authorized Step P3 implementation.

VERIFICATION

-

#27 Render pages from data

Working prompt | 2026-10-16

PROMPT

```
Replace the hard-coded sample rows in the dashboard, expenses, income, budgets, recurring, categories, groups, group detail, settlements, shopping and profile pages with markup rendered by JavaScript from api.js, using <template> elements and textContent (never innerHTML with user data). Keep the static markup as the no-JavaScript fallback. Implement the expenses filter bar (search, category, method, date range, sort) with debounced live filtering and query-string sync, pagination, and empty states.
```

PURPOSE

Turn the static Review 1 HTML pages into interactive screens rendered from the Review 2 client data layer, adhering to strict XSS safety and accessible empty/loading states.

FILES

client/js/pages/dashboard.js, client/js/pages/expenses.js, client/js/pages/income.js, client/js/pages/budgets.js, client/js/pages/recurring.js, client/js/pages/categories.js, client/js/pages/groups.js, client/js/pages/group-detail.js, client/js/pages/settlements.js, client/js/pages/shopping.js, client/js/pages/shopping-detail.js, client/js/pages/profile.js, client/js/pages/analytics.js, client/js/pages/expense-edit.js, client/js/core/render.js, client/js/ui/user.js, client/js/app.js, client/pages/profile.html, client/README.md, tests/client/render.test.mjs

RESULT

Created 14 modular page scripts in client/js/pages/ importing exclusively from api.js; implemented live debounced expenses filter bar with URL sync and pagination; created core rendering helpers for safe XSS-free

DOM creation, aria-busy loading states, and accessible empty/error states; wired user profile display and reset demo data; added comprehensive rendering test suite.

VERIFICATION

All 47 unit/integration tests passing (30 data layer + 17 rendering/UI); npm run lint:html passing with 0 errors across all 25 HTML files.

#28 Request: proceed with Review 2 Step P4

User prompt | 2026-10-16

PROMPT

```
Proceed to P4: Forms, validation and CRUD flows. First, inspect the P4 instructions in the handover PDF and review the existing page modules, client/js/data/api.js, client/js/core/, client/js/ui/, and the current HTML forms. Implement: 1. Connect the login and registration forms to the existing authentication API. 2. Connect expense creation and editing, income creation, group creation, shared-expense creation, shopping-list creation, and other forms specified by P4. 3. Implement client-side validation with accessible field-level errors, appropriate required fields, valid dates, valid integer-paise amounts, and clear error messages. 4. Convert rupee values entered by users to integer paise at the API boundary. Never store floating-point monetary amounts. 5. Disable submit buttons while saving, prevent accidental duplicate submissions, and show success messages only after the operation succeeds. 6. Preserve form values when validation or API operations fail. 7. Use the existing confirmation dialog for destructive operations and ensure cancellation never changes data. 8. Handle missing or invalid record IDs, duplicate registrations, unauthorized operations, and API errors gracefully. 9. Keep page modules importing data operations exclusively from client/js/data/api.js. Do not access store.js directly from page modules. 10. Add automated tests for successful submissions, validation errors, duplicate submission prevention, edit flows, delete confirmation, and failure handling. Do not implement the P6 analytics features, P7 receipt OCR, or backend integration early. Run all relevant tests, including the data-layer and rendering tests, and npm run lint:html. Update documentation and the prompt log. Review the diff and commit locally on refurbish using the handover's required commit message, or P4: forms validation and CRUD if none is specified. Report completed forms, supported operations, test results, known limitations, and commit hash.
```

PURPOSE

Authorize execution of Step P4 to connect HTML forms to api.js, implement client-side validation, duplicate submission guards, and complete CRUD workflows.

RESULT

Authorized Step P4 implementation.

VERIFICATION

-

#29 Forms, validation and CRUD flows

Working prompt | 2026-10-16

PROMPT

```
Wire every form (register, login, expense add/edit, income, budget, recurring, category, group create, add member, shared expense, shopping list and item, profile, change password) to api.js. Use the Constraint Validation API plus custom rules (passwords match, date not in the future, budget greater than zero) with inline error messages linked by aria-describedby, and move focus to the first invalid field. Show a toast after each save and redirect afterwards (Post/Redirect/Get). Add password show/hide toggles with aria-pressed. Protect app pages: redirect to login.html when nobody is signed in.
```

PURPOSE

Cover the core JavaScript skills examiners look for: events, DOM manipulation, validation, and full CRUD workflows.

FILES

client/js/ui/forms.js, client/js/app.js, client/js/data/api.js, client/js/pages/login.js, client/js/pages/register.js, client/js/pages/expense-form.js, client/js/pages/expense-edit.js, client/js/pages/expenses.js, client/js/pages/income-form.js, client/js/pages/income.js, client/js/pages/group-create.js, client/js/pages/group-detail.js, client/js/pages/shared-expense-form.js, client/js/pages/shopping-form.js, client/js/pages/shopping-detail.js, client/js/pages/budgets.js, client/js/pages/recurring.js, client/js/pages/categories.js, client/js/pages/profile.js, tests/client/forms.test.mjs

RESULT

Implemented reusable form validation and accessible error UI helper module (client/js/ui/forms.js) with rupeesToPaise and setButtonLoading; added authGuard in app.js; created form page scripts for login, register, expense create/edit, income create, group create, shared expense with equal/custom splits, shopping list create; wired CRUD actions across existing pages (delete expenses/income with confirmDialog, budget create/remove, recurring create/toggle/delete, category create/delete, group add-member/leave, shopping items/convert-to-expense, profile update/change-password); added comprehensive automated test suite (30 passing tests).

VERIFICATION

77 total client tests passing across data-layer.test.mjs, render.test.mjs, and forms.test.mjs; npm run lint:html passing with 0 errors across 25 HTML files.

#30 Request: proceed with Review 2 Step P5

User prompt | 2026-10-16

PROMPT

```
# SplitStack — P5: Group Splitting and Debt Simplification You are continuing development of SplitStack, a finance and expense-sharing application. Steps P1–P4 are reported complete. Implement only Step P5: Group Splitting and Debt Simplification. 1. Mandatory preflight: read P5 instructions in SplitStack_Handover_Prompts.pdf; inspect api.js, store.js, seed.js, groups.js, group-detail.js, settlements.js, shared-expense-form.js, relevant HTML pages, and existing tests; confirm public API signatures and data schema; preserve docs/audit/. 2. Group expense calculations: handle equal splits with deterministic remainder paise allocation, custom splits with explicit member shares, payer and participant validation, integer-paise conservation, and deletion/editing recalculations. 3. Net balances: implement single authoritative balance-calculation function (calculateNetBalances) with sign convention (positive = creditor / to receive, negative = debtor / owes, zero = settled), zero-sum invariant across group. 4. Minimum-cash-flow debt simplification: implement greedy approach matching largest debtor with largest creditor, transferring min(debt, credit), positive integer paise, stable deterministic output, at most N-1 transfers. 5. Settlement recording: validate group membership, distinct payer/payee, positive integer paise, cannot exceed outstanding debt, duplicate debounce, balance recalculation. 6. Page integration: connect group-detail and settlements pages to canonical calculation and debt simplification logic with confirmation dialogs and toast feedback. 7. Automated tests: add comprehensive test suite for splits, balances, simplification, and settlements; run regression suite and HTML lint; update documentation and prompt log; commit locally with message: Review 2: group splitting and debt simplification.
```

PURPOSE

Authorize implementation of Step P5: Group Splitting and Debt Simplification.

RESULT

Authorized Step P5 implementation.

VERIFICATION

-

#31 Group splitting and debt simplification

Working prompt | 2026-10-16

PROMPT

```
In client/js/domain/splits.js implement equal splits in integer paise with the leftover paise assigned to the payer, and custom splits that must sum exactly to the total. On the shared-expense form show a live "remaining to assign" counter and disable submit until it reaches zero. In client/js/domain/settle.js implement net balances (paid minus share) and the greedy minimum-cash-flow algorithm: repeatedly match the largest debtor with the largest creditor and transfer min(debt, credit) until every balance is within one paisa; return the transfer list and the number of original pairwise debts. Render the settle-up page from it, and make "Mark paid" record a settlement and recompute balances. Add unit tests runnable with node --test using the Goa Trip example (balances +1700, -2500, +5700, -4900 give 3 transfers).
```

PURPOSE

Implement authoritative group expense accounting, deterministic integer-paise remainder allocation, greedy minimum-cash-flow debt simplification, and settlement validation.

FILES

client/js/domain/splits.js, client/js/domain/settle.js, client/js/data/api.js, client/js/pages/group-detail.js, client/js/pages/shared-expense-form.js, client/js/pages/settlements.js, tests/client/splits-settle.test.mjs, client/README.md

RESULT

Created domain modules splits.js (equal split with deterministic remainder paise to payer, custom shares validation) and settle.js (canonical calculateNetBalances with strict zero-sum conservation invariant, greedy simplifyDebts minimizing payments to at most N-1); integrated into api.js (createSharedExpense, updateSharedExpense, getGroupBalances, getGroupMemberStats, getSuggestedSettlements, recordSettlement with debt-limit validation, removeSettlement); wired live remaining tracker in shared-expense-form.js; updated group-detail.js to show member paid/share/net and suggested settlements; updated settlements.js with confirmDialog, toast notifications, loading states, and settlement removal; added 27 comprehensive automated tests.

VERIFICATION

104 total client tests passing across 4 test suites (data-layer.test.mjs, render.test.mjs, forms.test.mjs, splits-settle.test.mjs); npm run lint:html passing with 0 errors across 25 HTML files.

#32 Request: proceed with Review 2 Step P6

User prompt | 2026-10-16

PROMPT

```
# SplitStack — P6: Analytics, Charts, Forecasts, Insights, Search & Notifications You are
continuing development of SplitStack, a personal finance and expense-sharing application.
P1–P5 are reported complete. Implement only P6, following the project handover instructions.
1. Mandatory preflight: read P6 in SplitStack_Handover_Prompts.pdf; inspect project structure,
REFURBISH_PLAN.md, analytics/budget/recurring/search/notification modules, api.js, store.js,
seed.js, splits.js, settle.js, analytics.js, budgets.js, dashboard.js, HTML/CSS, and tests;
verify git branch refurbish and latest commit; preserve docs/audit/. 2. Scope and boundaries:
implement financial analytics/charts, budget tracking/forecasting, explainable spending
insights, search/filter improvements, and in-app notifications. Do not implement P7 OCR or
backend P9+. 3. Analytics data layer: improve reusable functions behind api.js; keep
calculations independent of DOM; integer paise for stored amounts; respect active user and
data isolation; distinguish personal transactions from group expenses; handle sparse data. 4.
Financial analytics and charts: implement visualizations using real data (trends, category
spend, net savings, budget usage); dynamic updates on filter changes; accessible
titles/legends/tables; prevent duplicate chart instances. 5. Budget tracking and forecasting:
linear spending velocity projection in domain/forecast.js; minimum 3 transactions data
sufficiency rule; explainable estimates; risk classification (danger, warning, caution, ok).
6. Spending insights: deterministic insights from real data in domain/insights.js (budget
danger/warning, >=20% month-over-month category trend, upcoming recurring bills in 7 days,
group balances). 7. Search and filtering: topbar global combobox search in ui/search.js with
200ms debounce across expenses, income, groups, shopping lists; case-insensitive; clear
filters restoration; pagination consistency. 8. In-app notifications: topbar bell indicator in
ui/notifications.js; dropdown panel; read status persistence per user in store.js. 9.
Automated tests: add forecast, insights, and search/notification tests; run all client tests;
run npm run lint:html; update documentation and prompt log; commit locally with message:
Review 2: charts, budgets, forecast, insights, search, notifications.
```

PURPOSE

Authorize implementation of Step P6: Analytics, Charts, Forecasts, Insights, Search & Notifications.

RESULT

Authorized Step P6 implementation.

VERIFICATION

-

#33 Charts, budgets, forecast, insights, search and notifications

Working prompt | 2026-10-16

PROMPT

```
Render the dashboard and analytics charts with Chart.js 4 loaded from a pinned CDN URL,
reading colours from the CSS custom properties so both themes work, and keep the CSS charts as
a fallback. Compute budget status (ok under 75%, caution 75-90%, warning 90-100%, danger over
100%) and update the meters. Port the spending forecast (average daily spend multiplied by the
days in the month) to client/js/domain/forecast.js. Generate smart insights from the data
(over-budget categories, month-over-month change, upcoming recurring bills). Make the topbar
search show live results across expenses, groups and lists in a listbox with keyboard
navigation, and make the bell open a notifications panel built from the insights with an
unread count.
```

PURPOSE

Finish the interactive features that were disconnected in the original app.

FILES

client/js/domain/forecast.js, client/js/domain/insights.js, client/js/ui/charts.js, client/js/ui/search.js, client/js/ui/notifications.js, client/js/pages/dashboard.js, client/js/pages/analytics.js, client/js/pages/budgets.js, client/js/data/api.js, client/js/app.js, client/css/components.css, package.json, tests/client/forecast.test.mjs, tests/client/insights.test.mjs, tests/client/search-notifications.test.mjs, client/README.md

RESULT

Created forecast.js domain module with daily velocity model, data sufficiency threshold (>= 3 transactions), and budget risk classification; created insights.js domain module with deterministic budget alerts, month-over-month category trend tracking (>= 20%), upcoming recurring bills (next 7 days), and group balance alerts; created charts.js with Chart.js 4 dynamic loader, token palette extraction via getComputedStyle, theme observer, canvas aria-label, and CSS chart fallbacks; created search.js with 200ms debounce, WAI-ARIA combobox/listbox pattern, and keyboard navigation across expenses, income, groups, and shopping lists; created notifications.js with topbar bell badge and dropdown panel with read-state persistence in user preferences; updated dashboard.js, analytics.js, budgets.js, and app.js; added test:client script to package.json; added 41 new automated tests.

VERIFICATION

145 total client tests passing across 7 test suites (data-layer.test.mjs, render.test.mjs, forms.test.mjs, splits-settle.test.mjs, forecast.test.mjs, insights.test.mjs, search-notifications.test.mjs); npm run lint:html passing with 0 errors across 25 HTML files.

#34 Request: proceed with Review 2 Step P7

User prompt | 2026-10-17

PROMPT

```
# SplitStack — P7: Shopping Checklist & Receipt Scanning Preview You are working on the existing SplitStack repository in VS Code using Antigravity. ## Objective Complete **P7 only** from the SplitStack roadmap: implement and test the shopping checklist experience and receipt scanning preview, building on the existing P1–P6 implementation. Do not start P8 or any later phase. Finish P7 end-to-end, run the relevant tests, update the project logs, and create a local Git commit only after verification succeeds. 1. Safety checks before editing 2. Inspect the existing P7-related implementation 3. Shopping checklist functionality 4. Receipt scanning preview 5. Security and data integrity 6. Tests 7. Verification 8. Documentation and prompt log 9. Review and local commit 10. GitHub restrictions 11. Final report
```

PURPOSE

Authorize execution of Step P7: Shopping Checklist & Receipt Scanning Preview.

RESULT

Authorized Step P7 implementation.

VERIFICATION

-

#35 Shopping checklist and receipt scan preview

Working prompt | 2026-10-17

PROMPT

```
Complete the shopping checklist experience and client-side receipt scanning preview: enable shopping list create/edit/delete, item CRUD, inline item editing (name, quantity, estimated and actual unit price), optimistic checkbox toggling with rollback on error, confirmation dialogs for item removal and list deletion, progress calculation, and duplicate-protected conversion to expense. Implement client-side receipt scanning preview: drag & drop dropzone with .is-dragover styling, file validation (PNG, JPEG, WEBP <= 5 MB), in-memory object URL preview with revokeObjectURL cleanup, cancel/reset option, honest OCR status notice (client preview mode with automated OCR deferred to Review 3), candidate receipt parsing, and editable review step with Discard and Confirm-and-save to expense.
```

PURPOSE

Deliver the shopping checklist lifecycle and client-side receipt scanning preview within Review 2.

FILES

client/js/domain/receipt.js, client/js/pages/receipt-scan.js, client/js/pages/receipt-confirm.js, client/js/pages/shopping-form.js, client/js/pages/shopping-detail.js, client/pages/shopping-detail.html, client/js/data/api.js, client/css/components.css, package.json, tests/client/shopping-receipt.test.mjs, tests/client/render.test.mjs, client/README.md

RESULT

Created domain module receipt.js with file validation (MIME, extension, 5 MB limit), formatFileSize, sanitizeReceiptFilename, candidate text parsing (merchant, date, amountPaise), and honest OCR availability check; created receipt-scan.js with drag & drop, safe object URL preview, metadata box, cancel/reset, and session transition; created receipt-confirm.js with preview rendering, honest OCR status alert, editable review form, Discard handler, and saving to expense & receipt collections; updated shopping-form.js to support both create and edit modes; updated shopping-detail.js with inline item editing, optimistic checklist toggling with error rollback, confirmDialog on item removal, and duplicate conversion guard; updated shopping-detail.html with Edit list button and progress meter; updated api.js with enhanced shopping validation and listReceipts, getReceipt, createReceipt methods; added 31 new automated tests in shopping-receipt.test.mjs.

VERIFICATION

176 total client tests passing across 8 test suites (data-layer.test.mjs, render.test.mjs, forms.test.mjs, splits-settle.test.mjs, forecast.test.mjs, insights.test.mjs, search-notifications.test.mjs, shopping-receipt.test.mjs); npm run lint:html passing with 0 errors across 25 HTML files.

#36 Request: proceed with Review 2 Step P8

User prompt | 2026-10-17

PROMPT

```
# P8 — Review 2: QA, Final Audit, and Commit Continue the SplitStack project from the current refurbish branch. P1–P7 are reported complete. This phase is P8 only. ## 1. Safety and repository rules - Work only on the refurbish branch. - First inspect git status, git branch --show-current, git log, and the latest commit. - Do not switch branches, reset, rebase, amend existing commits, or discard changes. - Preserve the untracked docs/audit/ directory and all of its contents. Inspect it before making changes; never delete or overwrite existing audit evidence without a clear reason. - Preserve every legacy Flask file and directory, including app.py, config.py, Procfile, requirements.txt, modules/, templates/, static/, database/, ml/, and legacy tests. - Do not start P9 or implement Express, MongoDB, Mongoose, authentication, server APIs, or server OCR. - Do not push to any remote. - Do not install dependencies or change lockfiles unless strictly necessary and explicitly justified. - Do not claim a test or browser check passed unless you actually ran it and observed the result. ## 2. Audit the completed work Review the current implementation and existing tests for P1–P7, especially: - Module architecture and application shell. - Data persistence, recovery, and logout behavior. - Rendering, forms, validation, and CRUD operations. - Group expense splitting and debt simplification. - Analytics, charts, budgets, forecasts, insights, search, and notifications. - Shopping checklist interactions and receipt-upload preview/review flow. - API/data-layer contracts, error handling, and regression risks. - Security, accessibility, and responsive behavior where applicable. - Preservation of the legacy Flask application. ## 3. Run verification Inspect package.json first and use the existing scripts. Run the relevant checks, including: - npm run test:client - npm run lint:html - Legacy Python tests, including tests/test_auth_flow.py, using the project's existing environment and configuration. ## 4. Fix issues carefully ## 5. Documentation and audit evidence ## 6. Commit criteria Use the commit message: Review 2: QA audit and regression verification Do not push. ## 7. Final report
```

PURPOSE

Authorize execution of Step P8: Review 2 QA audit, full regression verification, documentation, and checkpoint commit.

RESULT

Authorized Step P8 implementation and audit.

VERIFICATION

-

#37 Review 2 QA, final audit and regression verification

Working prompt | 2026-10-17

PROMPT

```
Conduct comprehensive QA audit across completed Review 2 steps P1–P7: verify module architecture and application shell; verify data persistence, recovery, and logout behavior; verify rendering, forms, validation, and CRUD operations; verify group splitting and debt simplification; verify analytics, charts, budgets, forecasts, insights, search, and notifications; verify shopping checklist interactions and receipt-upload preview/review flow; verify API/data-layer contracts, error handling, and regression risks; verify security, accessibility, and responsive behavior; verify preservation of the legacy Flask application. Run automated client test suite, HTML linter, legacy Python test suite, document findings in docs/audit/P8_REVIEW_2_QA_AUDIT.md, update client/README.md with JavaScript architecture and concepts summary, update prompt logs, and commit with message 'Review 2: QA audit and regression verification'.
```

PURPOSE

Close Review 2 with comprehensive evidence, complete test execution results, architectural review, and regression verification.

FILES

client/README.md, docs/audit/P8_REVIEW_2_QA_AUDIT.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

All 176 client tests passing across 52 test suites; HTML-validate passing on all 25 pages with 0 errors; legacy Python tests passing (test_master.py 100% Master Pass, test_auth_flow.py 8/8 pass, test_security.py 10/10 pass, test_regression.py 12/12 pass); documented QA findings in docs/audit/P8_REVIEW_2_QA_AUDIT.md; added Review 2 JavaScript Concepts & Architecture table to client/README.md.

VERIFICATION

npm run test:client (176 passed, 0 failed); npm run lint:html (0 errors); py -3 tests/test_auth_flow.py (8 passed); py -3 tests/test_master.py (8 phases passed); py -3 tests/test_security.py (10 passed); py -3 tests/test_regression.py (12 passed).

Review 3 - Node.js + MongoDB - Express REST API, Mongoose models, authentication, OCR

#38 Request: proceed with Review 3 Step P9

User prompt | 2026-10-17

PROMPT

```
# P9 — Review 3: Node.js and Express Backend Foundation Begin phase P9 only for the SplitStack project. ## Repository safety - Work only on the refurbish branch. - First inspect the branch, working tree, recent commits, package.json, existing client API contract, README files, and current directory structure. - Confirm that commit 82b648b is present before proceeding. - Do not push to any remote. - Do not reset, rebase, amend, or discard changes. - Preserve all legacy Flask files and directories, including app.py, config.py, Procfile, requirements.txt, modules/, templates/, static/, database/, ml/, and legacy tests. - Preserve all existing docs/audit/ evidence. - Do not implement P10–P15 in this phase. ## Objective Create the minimal, maintainable Node.js and Express server foundation that later phases can extend. ## Implementation - Establish a clearly separated Node.js server entry point. - Configure Express with appropriate JSON parsing, environment-based configuration, and a simple health-check endpoint. - Add sensible centralized error handling and a not-found response. - Use environment variables for configurable host/port settings; provide a safe example environment file if needed, without real secrets. - Keep development configuration separate from production assumptions. - Add a basic server test using the repository's existing test conventions where practical. - Document how to install dependencies, configure the environment, start the server, check its health endpoint, and run its tests. - Keep the implementation small and consistent with the current project architecture. ## Security and quality - Do not hardcode credentials or secrets. - Do not log passwords, tokens, or sensitive user data. - Do not enable permissive CORS without a demonstrated need; document any origin configuration. - Do not claim the server is production-ready. - Avoid unrelated frontend changes. - Do not connect to MongoDB yet unless an existing project requirement explicitly makes it necessary; database models and connection work belong to P10. - Do not implement authentication or REST resource endpoints yet; those belong to P11. - Do not implement OCR or receipt file storage; those belong to P12. ## Verification - Run the relevant server tests and existing client tests. - Run the HTML linter if any frontend-facing files change. - Report exact commands, observed outcomes, and any blocked checks. - Inspect the complete diff and ensure legacy Flask files remain untouched. ## Documentation and commit - Update the appropriate README and prompt log using the established project workflow. - Add a P9 audit entry while preserving existing audit files. - Stage only reviewed P9 files and intended documentation. - Commit only after reviewing the diff and checking for accidental secrets or unrelated changes. - Commit message: Review 3: Express server foundation - Do not push. ## Final report
```

PURPOSE

Authorize execution of Step P9: Node.js and Express backend foundation.

RESULT

Authorized Step P9 implementation.

VERIFICATION

-

#39 Express server foundation

Working prompt | 2026-10-17

PROMPT

```
Create server/ as a Node.js 20+ project (ES modules) with Express: server/package.json with minimal dependencies (express, dotenv) and scripts start, dev, test; server/src/config.js with environment configuration, default port 3000, host localhost, and validation; server/src/app.js with createApp factory, express.json(limit: 100kb), safe request logging, health-check endpoint GET /api/health returning status, uptime, timestamp, environment, service, version, static client asset serving, catch-all 404 handler returning JSON, and centralized error handling middleware; server/src/index.js entry point with graceful SIGINT/SIGTERM shutdown; server/.env.example with safe configuration template; server/tests/server.test.js with automated tests for health check, 404 handler, JSON parsing, error formatting, and static frontend; server/README.md; add server and test:server scripts to root package.json; document P9 audit in docs/audit/P9_EXPRESS_FOUNDATION_AUDIT.md and update prompt log.
```

PURPOSE

Establish the minimal, maintainable Node.js and Express backend foundation for Review 3.

FILES

server/package.json, server/package-lock.json, server/.env.example, server/src/config.js, server/src/app.js, server/src/index.js, server/tests/server.test.js, server/README.md, package.json, README.md, docs/audit/P9_EXPRESS_FOUNDATION_AUDIT.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

Created clean Express server foundation in server/ with health check at /api/health, centralized 404/error handling, 100kb JSON parsing limit, disabled X-Powered-By header, and static frontend hosting; all 7 server tests passing in node --test; live health check verified; root package.json and README.md updated.

VERIFICATION

npm run test:server (7 passed, 0 failed); npm run test:client (176 passed, 0 failed); npm run lint:html (0 errors); live GET /api/health returned 200 OK with expected JSON payload.

#40 Request: proceed with Review 3 Step P10

User prompt | 2026-10-17

PROMPT

```
# P10 — Review 3: MongoDB Connection and Mongoose Data Models Continue the SplitStack project from the current refurbish branch. P9 is reported complete with local commit 30a4d60. Implement P10 only. ## 1. Safety rules - Verify the current branch, working tree, and latest commit before editing. - Work only on refurbish. - Do not push to any remote. - Do not reset, rebase, amend commits, or discard changes. - Preserve the complete legacy Flask application and all existing docs/audit/ evidence. - Do not implement P11 authentication or REST resource endpoints, P12 OCR/file storage, or P13 frontend API integration. - Never expose or commit real database credentials, secrets, or production connection strings. ## 2. Inspect before coding - Read server/package.json, server/src/config.js, server/src/app.js, existing tests, and the client API/data contracts. - Check the existing roadmap and project documentation for required data entities and relationships. - Determine which models are explicitly supported by the project requirements. - Review existing dependencies and lockfiles before making changes. ## 3. MongoDB configuration - Add Mongoose as a server dependency using a compatible version and update the server lockfile. - Implement a dedicated database connection module. - Read the MongoDB connection URI from an environment variable such as MONGODB_URI. - Update server/.env.example with a clearly fake local-development URI and any required database settings. - Validate configuration and produce safe, actionable errors without printing secrets. - Make database connection and disconnection functions explicit and testable. - Ensure importing the Express app does not automatically require a live database connection. - Update server startup to connect to MongoDB before accepting requests. If the connection fails, log a sanitized error and exit nonzero. Handle graceful shutdown without leaving connections open. - Do not claim a database connection succeeded unless it was actually tested against a reachable MongoDB instance. ## 4. Mongoose models - Define schemas and models for the entities required by the documented SplitStack domain. - Start by mapping existing documented entities: users, groups, expenses, participants/splits, settlements, budgets, recurring expenses, shopping lists/items, and receipts. - Avoid redundant models where embedded documents are a better fit. Explain major schema decisions. - Add appropriate field types, required constraints, enums, timestamps, indexes, and references where justified. - Validate monetary fields and document whether stored values use integer paise, consistent with the existing client contract. - Keep sensitive user fields out of ordinary serialization and never include passwords or secrets in logs. ## 5. Tests - Add tests for schema validation, required fields, enum constraints, relationships/references, and monetary validation. - Prefer isolated tests that do not require an external MongoDB instance. - Run npm run test:server and npm run test:client. - Run npm run lint:html if any frontend files change. ## 6. Documentation and audit - Update server/README.md with database configuration, model overview, startup behavior, and testing instructions. - Add a P10 audit report in docs/audit/. - Update the prompt log and regenerate its DOCX/PDF using the existing npm run log workflow. ## 7. Commit - Commit message: Review 3: MongoDB connection and Mongoose models - Do not push. ## 8. Final report
```

PURPOSE

Authorize execution of Step P10: MongoDB connection lifecycle and 12 Mongoose data models.

RESULT

Authorized Step P10 implementation.

VERIFICATION

-

#41 MongoDB connection and Mongoose data models

Working prompt | 2026-10-17

PROMPT

```
Install mongoose in server/; update server/src/config.js with mongodbUri; create
server/src/db.js with connectDb, disconnectDb, sanitizeMongoUri, and readyState tracking;
update server/src/app.js health check with database connection status; update
server/src/index.js to connect to MongoDB before listening and handle graceful shutdown;
create server/src/models/validators.js for integer paise and RFC email validation; create 12
Mongoose models in server/src/models/ (User, Category, Expense, Income, Budget, Recurring,
Group, SharedExpense, Settlement, ShoppingList, Receipt, Insight) matching client data
contracts with integer-paise validation, embedded subdocuments (members, shares, checklist
items), and toJSON password stripping; create server/tests/models.test.js with 31 unit tests
for schema validation, constraints, and URI sanitization; update server/README.md and
server/.env.example; document P10 audit in docs/audit/P10_MONGOOSE_MODELS_AUDIT.md and update
prompt logs.
```

PURPOSE

Implement the persistent MongoDB connection lifecycle and 12 Mongoose data models matching the SplitStack domain.

FILES

server/package.json, server/package-lock.json, server/.env.example, server/src/config.js, server/src/db.js, server/src/app.js, server/src/index.js, server/src/models/validators.js, server/src/models/User.js, server/src/models/Category.js, server/src/models/Expense.js, server/src/models/Income.js, server/src/models/Budget.js, server/src/models/Recurring.js, server/src/models/Group.js, server/src/models/SharedExpense.js, server/src/models/Settlement.js, server/src/models/ShoppingList.js, server/src/models/Receipt.js, server/src/models/Insight.js, server/src/models/index.js, server/tests/models.test.js, server/README.md, docs/audit/P10_MONGOOSE_MODELS_AUDIT.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

Created database connection manager and 12 validated Mongoose models with integer-paise monetary enforcement; all 38 server tests passing across 19 suites; client test suite (176 tests) and HTML linter (25 files) clean; documentation updated.

VERIFICATION

npm run test:server (38 passed, 0 failed); npm run test:client (176 passed, 0 failed); npm run lint:html (0 errors).

#42 Request: proceed with Review 3 Step P11

User prompt | 2026-10-17

PROMPT

```
# P11 — Review 3: Authentication and REST API Endpoints Continue the SplitStack project from the current refurbish branch. P10 is reported complete at commit 313f69f. Implement P11 only. ## 1. Repository safety - Verify the branch, working tree, latest commit, and existing server architecture before editing. - Work only on refurbish. - Do not push to any remote. - Do not reset, rebase, amend commits, or discard changes. - Preserve all legacy Flask files, existing audit evidence, and unrelated untracked files. - Do not implement P12 OCR/file upload, P13 frontend API integration, or later phases. - Do not claim live database integration works unless tested against a reachable MongoDB instance. ## 2. Review existing contracts first Inspect server files, Mongoose models, client API contracts, and roadmap reports. ## 3. Authentication Implement secure registration, login, logout, and current-user/session endpoints. - Use bcrypt (cost factor 12). Never store plaintext passwords. - Never return password hashes in API responses. - Validate and normalize inputs; enforce unique email addresses. - Use generic login failure messages that do not reveal whether an email exists. - Apply request-size limits and authentication rate limiting. - Use secure session/token handling (JWT, HttpOnly cookie with SameSite: Lax). - Never log passwords, tokens, cookies, or credentials. - Authorization middleware derives authenticated user from verified credentials. ## 4. REST API Implement documented P11 endpoints and controllers for core SplitStack resources. - Consistent success and error response formats. - Request validation and appropriate HTTP status codes. - Enforce per-user ownership and group membership/role checks on every operation. - Validate referenced document IDs (reject malformed ObjectIds). - Scoped Mongoose queries; avoid leaking internal stack traces. ## 5. Database and tests - Keep Express app independently importable for tests without automatically connecting to MongoDB. - Add authentication tests (registration, hashing, login success/failure, logout, rate limiting). - Add API tests (authorization, ownership isolation, input validation, correct responses). - Run npm run test:server and npm run test:client. - Run npm run lint:html if frontend files change. ## 6. Documentation and audit - Update server/README.md and .env.example. - Add P11 audit report in docs/audit/. - Update prompt log and regenerate DOCX/PDF via npm run log. ## 7. Commit - Commit message: Review 3: authentication and REST API - Do not push. ## 8. Final report
```

PURPOSE

Authorize execution of Step P11: Authentication and REST API Endpoints.

RESULT

Authorized Step P11 implementation.

VERIFICATION

-

#43 Authentication and REST API endpoints

Working prompt | 2026-10-17

PROMPT

```
Install bcryptjs, jsonwebtoken, and cookie-parser in server/; update server/src/config.js with JWT and cookie configuration; update server/.env.example with safe security templates; create server/src/utils/errors.js and server/src/utils/domain.js; create server/src/services/authService.js with bcrypt hashing (cost factor 12), JWT generation, email normalization, and generic login protection; create server/src/middleware/auth.js, server/src/middleware/rateLimit.js (in-memory sliding window), and server/src/middleware/validate.js (ObjectId and paise checks); create REST resource routes in server/src/routes/ (auth, expenses, income, categories, budgets, recurring, groups, shopping, analytics, notifications, search, receipts); mount routes and cookie-parser in server/src/app.js; add comprehensive test suites in server/tests/auth.test.js and server/tests/api.test.js; update server/README.md and docs/audit/P11_AUTH_REST_API_AUDIT.md; regenerate prompt logs.
```

PURPOSE

Implement production-grade authentication and the complete set of REST API endpoints for SplitStack with ownership isolation and monetary validation.

FILES

server/package.json, server/package-lock.json, server/.env.example, server/src/config.js, server/src/app.js, server/src/utils/errors.js, server/src/utils/domain.js, server/src/middleware/auth.js, server/src/middleware/rateLimit.js, server/src/middleware/validate.js, server/src/services/authService.js, server/src/routes/auth.js, server/src/routes/expenses.js, server/src/routes/income.js, server/src/routes/categories.js, server/src/routes/budgets.js, server/src/routes/recurring.js, server/src/routes/groups.js, server/src/routes/shopping.js, server/src/routes/analytics.js, server/src/routes/notifications.js, server/src/routes/search.js, server/src/routes/receipts.js, server/src/routes/index.js, server/tests/auth.test.js, server/tests/api.test.js, server/README.md, docs/audit/P11_AUTH_REST_API_AUDIT.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

Created complete authentication layer and 12 REST API modules with strict ownership isolation, group role checks, and paise validation; all 88 server tests passing across 33 suites; client test suite (176 tests) and HTML linter (25 files) clean; documentation and audit updated.

VERIFICATION

npm run test:server (88 passed, 0 failed); npm run test:client (176 passed, 0 failed); npm run lint:html (0 errors).

#44 Request: proceed with Review 3 Step P12

User prompt | 2026-10-17

PROMPT

```
# SplitStack – P12: Receipt Upload and Server-Side OCR (Review 3) Implement P12 only of the SplitStack refurbishment roadmap on the existing refurbish branch. ## 1. Mandatory safety checks Before editing: Confirm current branch is refurbish, latest commit is c6c6f1d (Review 3: authentication and REST API), inspect git status. Work only on refurbish. Do not push. Preserve legacy Flask files and docs/audit/ evidence. ## 2. Inspect the existing implementation first Review server/src/app.js, server/src/config.js, middleware, routes, models, client receipt scanner, and package.json. ## 3. Implement secure receipt upload Implement POST /api/receipts/upload. Use Multer or equivalent. Accept explicitly supported receipt image formats (PNG, JPEG, WEBP) validated by content magic bytes, not just MIME type. Enforce 5 MB limit. Reject empty, malformed, or oversized files. Never trust client paths; generate server-controlled unique identifiers. Authenticate uploader and derive ownership exclusively from verified principal. Store in MongoDB GridFS with compensating cleanup on partial failures. ## 4. Implement server-side OCR Use tesseract.js to extract text server-side. Return structured candidate results (merchant, date, amount, confidence). Treat all extracted fields as untrusted suggestions. Confirming creates the expense; nothing becomes an expense without confirmation. OCR failures must produce safe, consistent errors without orphaned files. ## 5. Privacy and security Do not commit private receipt images or sensitive data. Use safe test fixtures. Enforce ownership on upload, download, metadata, and deletion. ## 6. Tests Add focused automated tests for upload, authentication, MIME sniffing, oversized files, OCR parsing, ownership, streaming, confirmation, and cleanup. ## 7. Documentation and audit Update server/README.md, .env.example, P12 audit report, and prompt_log.json. ## 8. Verification and commit Run test:server, test:client, lint:html. Commit message: Review 3: receipt upload and server-side OCR. Do not push. ## 9. Final report
```

PURPOSE

Authorize execution of Step P12: Receipt Upload and Server-Side OCR.

RESULT

Authorized Step P12 implementation.

VERIFICATION

-

#45 Receipt upload and server-side OCR

Working prompt | 2026-10-17

PROMPT

```
Install multer and tesseract.js in server/; update server/src/config.js and server/.env.example with MAX_RECEIPT_SIZE_BYTES, OCR_ENABLED, and OCR_LANGUAGE; create server/src/middleware/upload.js with 5 MB memory storage limit and validateImageSignature magic byte inspection for PNG, JPEG, and WEBP; create server/src/services/storageService.js with GridFS bucket management, server-controlled unique keys, streaming download, and compensating cleanup; create server/src/services/ocrService.js with Tesseract.js optical character recognition and candidate parsing via parseCandidateReceipt; update server/src/models/Receipt.js with mimeType, fileSizeBytes, and originalName; update server/src/routes/receipts.js with POST /upload, GET /, GET /:id, GET /:id/image (streaming), POST /:id/confirm (linked expense creation with duplicate 409 guard), and DELETE /:id (metadata and GridFS cleanup); update client/js/domain/receipt.js to prioritize explicit TOTAL line; create server/tests/receipts.test.js with 35 tests covering all upload, magic byte, OCR, ownership isolation, streaming, confirmation, and deletion behaviors; update server/README.md, docs/audit/P12_RECEIPT_UPLOAD_OCR_AUDIT.md, and prompt logs.
```

PURPOSE

Implement secure multipart receipt upload, binary magic byte content inspection, persistent GridFS storage, server-side OCR candidate extraction, and duplicate-protected confirmation for SplitStack.

FILES

client/js/domain/receipt.js, server/package.json, server/package-lock.json, server/.env.example, server/src/config.js, server/src/models/Receipt.js, server/src/middleware/upload.js, server/src/services/storageService.js, server/src/services/ocrService.js, server/src/routes/receipts.js, server/tests/receipts.test.js, server/README.md, docs/audit/P12_RECEIPT_UPLOAD_OCR_AUDIT.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

Implemented secure receipt upload pipeline with Multer and binary magic byte verification (PNG, JPEG, WEBP); MongoDB GridFS binary storage and streaming; server-side OCR via Tesseract.js with candidate extraction; duplicate-protected confirmation into linked Expense; all 123 server tests passing across 43 suites; client test suite (176 tests) and HTML linter (25 files) clean; documentation and audit updated.

VERIFICATION

npm run test:server (123 passed, 0 failed); npm run test:client (176 passed, 0 failed); npm run lint:html (0 errors).

#46 Request: proceed with Review 3 Step P13

User prompt | 2026-10-17

PROMPT

```
# SplitStack – P13: Frontend API Integration (Review 3) Implement P13 only of the SplitStack refurbishment roadmap on the existing refurbish branch. ## 1. Mandatory safety checks Before editing: Confirm branch refurbish and expected starting commit eef27b7 (Review 3: receipt upload and server-side OCR). Run git status and inspect recent commits. Do not push to any remote. Do not reset, rebase, amend commits, or discard existing work. Preserve all existing files under docs/audit/, including untracked evidence. Stage only intended files. Preserve the legacy Flask application and its files. Work only on refurbish. Do not begin P14 or P15. ## 2. Inspect the existing architecture first Review client/js/data/api.js, store abstraction, page rendering, session behavior, server routers, receipt scanning UI, READMEs, and prior audit reports. ## 3. Integrate the frontend data layer Replace or extend existing client API/data adapter so UI can use backend REST API while preserving page and domain contracts. Centralize API base URL and fetch behavior. Send authentication credentials supported by existing server. Do not store passwords or long-lived authentication tokens in localStorage. Handle HTTP errors, network failures, expired sessions, and validation errors consistently. Avoid silently treating failed API requests as successful empty data. Keep currency amounts in integer paise. Prevent duplicate form submissions. ## 4. Authentication and session behavior Integrate registration, login, logout, current-user, profile, and password-change endpoints. Restore session state using server's current-user endpoint. Do not expose passwords or tokens in UI state, logs, or persisted browser storage. ## 5. Resource integration Connect expenses, income, categories, budgets, recurring items, groups, members, shared expenses, balances, settlements, shopping lists, analytics, forecast, insights, notifications, search, and receipts. Match existing server route contracts exactly. ## 6. Data migration and offline behavior Do not silently discard or overwrite existing localStorage data. Provide deliberate, recoverable migration flow. Never fall back to local data after a server error in a way that falsely implies data was saved to the server. ## 7. Receipt upload integration Connect receipt scan/review UI to backend: upload image as multipart form data with field name receipt, do not manually set multipart boundary header, display upload/OCR progress, show OCR candidates as unverified suggestions, confirm expense only after user confirmation, display failures clearly and allow safe retry, prevent duplicate expenses. ## 8. Testing Add or update tests for base URL, JSON/multipart requests, HTTP/network errors, authentication restoration, resource payloads, receipt upload/review, and prevention of silent fallback. Run npm run test:client, npm run test:server, npm run lint:html. ## 9. Documentation and audit Update client README, docs/audit/P13_FRONTEND_API_INTEGRATION_AUDIT.md, and docs/prompt_log.json. Regenerate prompt-log DOCX/PDF via npm run log. ## 10. Final review and commit Commit with exact message: Review 3: frontend API integration. Do not push. ## 11. Final report
```

PURPOSE

Authorize execution of Step P13: Frontend API Integration.

RESULT

Authorized Step P13 implementation.

VERIFICATION

-

#47 Frontend API integration

Working prompt | 2026-10-17

PROMPT

```
Create client/js/data/api-http.js with centralized request() dispatcher, base URL management, query serialization, credentials: 'include', in-memory Bearer token management (tab-scoped sessionStorage), ApiError { field, message, status } mapping, and full resource methods for auth, expenses, income, categories, budgets, recurring, groups, shopping, analytics, search, receipts, and migration; update client/js/data/api.js with dual-mode dispatch (defaulting to 'api' in browser on http/https and 'local' in Node test runners); update client/js/pages/receipt-scan.js to upload receipts via POST /api/receipts/upload with button loading indicators; update client/js/pages/receipt-confirm.js to pre-fill OCR candidates as unverified suggestions, confirm via POST /api/receipts/:id/confirm with integer paise, handle 409 Conflict duplicate guards, and stream receipt image binary; update client/js/pages/group-detail.js and settlements.js with proper group context on deletion; create tests/client/api-http.test.mjs with 25 unit tests; update client/README.md, docs/audit/P13_FRONTEND_API_INTEGRATION_AUDIT.md, and prompt logs.
```

PURPOSE

Integrate the client data layer with the Express backend REST API while preserving existing page rendering and domain contracts without silent fallback or local data loss.

FILES

client/js/data/api-http.js, client/js/data/api.js, client/js/pages/receipt-scan.js, client/js/pages/receipt-confirm.js, client/js/pages/group-detail.js, client/js/pages/settlements.js, tests/client/api-http.test.mjs, client/README.md, docs/audit/P13_FRONTEND_API_INTEGRATION_AUDIT.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

Implemented client REST API client and dual-mode dispatching with zero silent fallback; HttpOnly credentials transport and in-memory tab-scoped Bearer token management; end-to-end receipt scanning and server OCR confirmation pipeline; 201 client tests passing across 59 suites (including 25 new tests in api-http.test.mjs); 123 server tests passing; HTML validation clean; documentation and audit updated.

VERIFICATION

npm run test:client (201 passed, 0 failed across 59 suites); npm run test:server (123 passed, 0 failed across 43 suites); npm run lint:html (25 files clean).

Wrap-up - Documentation, cleanup and deployment

#13 Request: GitHub repository, website link and PDF of prompts

User prompt | 2026-10-15

PROMPT

```
create a new repositry for this project in my gihub , and after than give me the github link and then the website link and also a pdf containing the prompts to complete project.
```

PURPOSE

Publish the project under the student's own GitHub account, host the site, and export the prompt log (including the remaining planned prompts) as a PDF.

RESULT

Done: see the following entries.

VERIFICATION

-

#14 PDF export of the prompt log, including planned prompts

Working prompt | 2026-10-15

PROMPT

```
Export the prompt log to PDF with a Python reportlab script (docs/tools/build_prompt_pdf.py)
that reads docs/prompt_log.json: a title page, a stage overview table, a summary of every
entry, Part A with every executed prompt verbatim in a shaded box with purpose, files, result
and verification, and Part B with the planned prompts that complete Review 2, Review 3 and the
wrap-up, clearly marked as not yet run. Add the planned prompts to prompt_log.json and to the
Word log as well, and make "npm run log" rebuild both documents.
```

PURPOSE

Give the student a PDF containing the prompts needed to complete the whole project.

FILES

docs/tools/build_prompt_pdf.py, docs/tools/build-prompt-log.js, docs/prompt_log.json, docs/SplitStack_Prompt_Log.pdf, docs/SplitStack_Prompt_Log.docx, package.json

RESULT

PDF with the executed prompts and 15 planned prompts; the Word log has the same content.

VERIFICATION

Pages rendered to images and inspected; text extracted with pypdf.

#15 Create the GitHub repository and push the code

Working prompt | 2026-10-15

PROMPT

```
Install the official GitHub CLI with winget and sign in through the device-code flow (the
student approves it in the browser). Before publishing, check that no secrets or databases are
tracked, review the receipt images that are still in the commit history (they are generic
sample invoices), and confirm with the student that a public repository is acceptable because
the team's original repository is private. Then create the public repository
MahaprabhuK/SplitStack, push the refurbish branch as main together with the legacy branch, add
a GitHub Actions workflow (.github/workflows/pages.yml) that deploys the client/ folder to
GitHub Pages, enable Pages with the "GitHub Actions" source, and confirm that the site
responds.
```

PURPOSE

Give the student a repository link and a live website link.

FILES

README.md

RESULT

Public repository https://github.com/MahaprabhuK/SplitStack created and the legacy branch pushed. The first push of main was rejected because it added a GitHub Actions workflow (.github/workflows/pages.yml) and the gh token lacked the "workflow" scope; the workflow file was removed and main was pushed in the next step. GitHub Pages is not set up yet (planned for the wrap-up; needs "gh auth refresh -s workflow" or a gh-pages branch).

VERIFICATION

gh repo view; git push output.

#16 Request: hand the project over to a teammate's agent

User prompt | 2026-10-15

PROMPT

```
ACTUALLY I GONNA HANDOVER THE PROJECT TO MY TEAMMATES AGENT SO HE'LL BE PULLING THE CODE AND RUNNING IN HIS SYSTEM SO GIVE PROMPT FOR THAT
```

PURPOSE

Let a teammate's AI agent clone, run and continue the project.

RESULT

A handover prompt was written covering: requirements per review, clone and branch, prerequisites, install/run/verify commands, repository map, known traps, the mandatory logging rule, the ordered remaining work (planned prompts P1-P15) and working rules. Noted that the new work had to be pushed to GitHub first.

VERIFICATION

-

#17 Push the refurbished code to GitHub main

Working prompt | 2026-10-15

PROMPT

```
push it (user prompt). Executed as: remove .github/workflows/pages.yml (the token lacks the workflow scope), replace the not-yet-existing live-site link in README.md with local run instructions, correct the earlier log entry about publishing, log the handover, rebuild the prompt logs, commit, and push the refurbish branch to main on https://github.com/MahaprabhuK/SplitStack so the teammate can clone it.
```

PURPOSE

Make the current work available to the teammate.

FILES

.github/workflows/pages.yml (removed), README.md, docs/prompt_log.json, docs/SplitStack_Prompt_Log.docx, docs/SplitStack_Prompt_Log.pdf

RESULT

main pushed to GitHub; branches on GitHub: main (refurbished work) and legacy (original Flask app).

VERIFICATION

git ls-remote shows main and legacy at the expected commits.

#18 Request: push the code and deploy the website

User prompt | 2026-10-15

PROMPT

```
first push the code and deploy it
```

PURPOSE

Get the code on GitHub and the Review 1 site online.

RESULT

Handled in the next step.

VERIFICATION

-

#19 Deploy the client to GitHub Pages from a gh-pages branch

Working prompt | 2026-10-15

PROMPT

```
Push refurbish to main and refurbish on MahaprabhuK/SplitStack. Because the token cannot push
workflow files, deploy without GitHub Actions: check that client/ uses only relative links,
split client/ into its own history with git subtree split, push it as the gh-pages branch,
wait for GitHub Pages to publish it, then put the live link back in README.md and log the step.
```

PURPOSE

Publish the Review 1 site at a public URL without needing the workflow scope.

FILES

README.md, docs/prompt_log.json

RESULT

main and refurbish are on GitHub. The client/ folder is published from the gh-pages branch at https://mahaprabhuk.github.io/SplitStack/ (GitHub enabled Pages automatically). To redeploy after client changes: git subtree split --prefix client refurbish, then push that commit to gh-pages.

VERIFICATION

git ls-remote; curl returned 200 for the home page, pages/404.html and css/base.css on the live site.

#20 Request: handover prompt PDF for a teammate's agent

User prompt | 2026-10-15

PROMPT

```
imma handover the project to my teammate i want you to create a detailed prompt for his agent
most prolly he'll use anti gravity so i want a pdf containing prompts to complete the rest of
the work in the project. so create a pdf and mention its path once its created then i'll share
it to him
```

PURPOSE

Let a teammate's AI agent (Antigravity) finish Review 2, Review 3 and the wrap-up.

RESULT

Handled in the next step.

VERIFICATION

-

#21 Handover prompts PDF

Working prompt | 2026-10-15

PROMPT

```
Write docs/tools/build_handover_pdf.py (reusing the styles of build_prompt_pdf.py) to produce
docs/SplitStack_Handover_Prompts.pdf: how to use it with Antigravity, one-time setup (tools,
collaborator access, MongoDB Atlas, workflow token scope), a roadmap, Prompt 0 with the full
project context, rules, logging, commit/push and gh-pages redeploy procedure, the 15 planned
prompts P1-P15 expanded with concrete checks and commit messages, utility prompts (status,
recovery, review rehearsal) and common problems. Render pages to check the layout.
```

PURPOSE

Give the teammate a self-contained, ordered set of prompts.

FILES

docs/tools/build_handover_pdf.py, docs/SplitStack_Handover_Prompts.pdf

RESULT

14-page PDF created. Changes against the planned prompts: P1 adds data-page attributes and client/js/package.json (type module) for node --test; P13 keeps the localStorage implementation as a demo mode (api-local.js chosen when /api/health does not answer) so the GitHub Pages site keeps working, instead of deleting it; P9 replaces deprecated csurf with a synchronizer token; P14 uses MongoMemoryReplSet because the API uses transactions; P15 asks for confirmation before deleting the Flask files and leaves Render account steps to the student.

VERIFICATION

Rendered pages with PyMuPDF and checked the layout of the setup, Prompt 0, step and utility pages.

Part B - Planned prompts to complete the project

Run these in order. Each builds on the previous one; after each, add a log entry and rebuild this document.

Review 3 - Node.js + MongoDB - Express REST API, Mongoose models, authentication, OCR

P1 Seed data and automated tests

Planned prompt

PROMPT

```
Add server/scripts/seed.js that creates the demo user, categories, six months of expenses and income, budgets, recurring payments, the three groups with shared expenses, and shopping lists. It must be idempotent and must refuse to run against a database whose name does not end in -dev or -test unless --force is passed. Write Jest + Supertest tests using mongodb-memory-server for auth, ownership checks, CSRF rejection, rate limiting, split validation, the settlement algorithm, budget status, convert-to-expense duplicate prevention and receipt upload validation, covering what the 47 legacy Python tests checked. Add an npm test script and a GitHub Actions workflow that runs lint and tests on every push.
```

PURPOSE

Prove the back end works and keep it working.

FILES

server/scripts/seed.js, server/tests/*.test.js, .github/workflows/ci.yml

Wrap-up - Documentation, cleanup and deployment

P2 Remove legacy code, document and deploy

Planned prompt

PROMPT

```
Delete the Flask app (app.py, config.py, database/, modules/, templates/, static/, ml/, tests/*.py, requirements.txt, Procfile) from main, since it is kept on the legacy branch. Rewrite README.md: features, the stack per review (HTML+CSS, JavaScript, Node.js+MongoDB), setup steps, environment variables, a data-model diagram, the API table and screenshots. Deploy the full app to Render as a Node web service (build: npm ci; start: npm start; environment: MONGODB_URI, SESSION_SECRET, NODE_ENV=production) with MongoDB Atlas network access configured, and keep the static site on GitHub Pages. Finish the prompt log, export it to Word and PDF, and tag the release v3.0.0.
```

PURPOSE

Deliver a clean, documented, deployed project.

FILES

README.md, render.yaml, docs/*